Interaktív feladatok

Kétfázisú (two steps / run twice) feladatok

Szerző: Nikházy László
Előadó: Szente Péter

2026. június 29.

Parrots (IOI 2011)

Yanee papagájokat tanít be, hogy egy `N` darab egész számból álló `M` üzenetet ($0 \leq$ `M[i]` ≤ 255) elküldjön egy távoli országba.

A probléma: A papagájok mind ugyanúgy néznek ki, és **tetszőleges sorrendben érkeznek meg** a célállomáshoz. Minden papagáj egyetlen egész számot képes megjegyezni 0 és `R` között. Legfeljebb `K` papagájt küldhetünk el összesen.

A feladat: Két különálló programot (függvényt) kell írnunk:

- `encode(N, M)`: Beolvassa az üzenetet, és a `send(a)` függvénnyel útnak indítja a papagájokat (legfeljebb `K` alkalommal, ahol $0 \leq$ `a` $\leq$ `R`).
- `decode(N, L, X)`: Fogadja az `L` darab megérkezett papagáj számait (`X` tömb, tetszőleges sorrendben), és a `output(b)` hívásokkal helyreállítja az eredeti üzenetet.

Fő kihívás: Hogyan vigyük át az értékeket úgy, hogy a megérkezési sorrend elveszik, azaz csak az értékek **multihalmazát** látjuk a túloldalon?

Részfeladatok és korlátok

A feladat pontszáma nagyban függ attól, mekkora az R (papagájok memóriája) és a K/N (papagáj/üzenet arány).

Részfeladat	N korlát	Értékkészlet	Max érték (R)	Max papagáj (K)
1 (17 pont)	$N = 8$	$M[i] \in \{0, 1\}$	65535	$10 \times N$
2 (17 pont)	$1 \leq N \leq 16$	$M[i] \in [0, 255]$	65535	$10 \times N$
3 (18 pont)	$1 \leq N \leq 16$	$M[i] \in [0, 255]$	255	$10 \times N$
4 (29 pont)	$1 \leq N \leq 32$	$M[i] \in [0, 255]$	255	$10 \times N$
5 (19 pont)	$16 \leq N \leq 64$	$M[i] \in [0, 255]$	255	$15 \times N$

A Subtask 5 pontozása (Skálázódó arány): Ha a használt papagájok aránya ($P = L/N$) legfeljebb 5, akkor jár a teljes 19 pont. Ha $5 < P \leq 6$, akkor 18 pont, és így tovább. Célunk a $P \leq 5$ elérése $N = 64$ és $R = 255$ mellett!

Subtask 1 & 2: Bit-pakolás (Bit-packing)

Paraméterek: $\texttt{R} = 65535$ (azaz 16 bit áll rendelkezésre), $\texttt{N} \leq 16$, $\texttt{K} = 10\texttt{N}$.

Az ötlet: Mivel $\texttt{R}$ nagyon nagy, egyetlen papagáj számába kényelmesen be tudjuk csomagolni mind az érték pozícióját (indexét), mind magát az értéket.

Kódolás: Minden $0 \leq \texttt{i} < \texttt{N}$ elemre küldünk pontosan egy papagájt:

$$\text{kód} = \texttt{i} \times 256 + \texttt{M[i]}$$

Mivel $\texttt{i} \leq 15$ és $\texttt{M[i]} \leq 255$, a maximális érték $15 \times 256 + 255 = 4351$, ami jóval kisebb, mint $\texttt{R} = 65535$.

Dekódolás: Bármilyen sorrendben is kapjuk meg a számokat, minden kapott $\texttt{X[j]}$ értékre:

- index = $\texttt{X[j]}/256$
- érték = $\texttt{X[j]} \pmod{256}$

Ezeket elmentjük egy tömbbe a megfelelő indexre, majd sorban kiírjuk.

Hatékonyság: Pontosan $\texttt{N}$ darab papagájt használtunk ($\texttt{P} = 1$). Ez 34 pontot ér!

Subtask 3 & 4: Bit-szintű küldés ($R = 255$)

Paraméterek: $R = 255$ (8 bit), $N \leq 32$, $K = 10N$. Most már nem fér el az index és az érték egymás mellett.

Az ötlet: Minden $M[i]$ szám 8 bites. Összesen legfeljebb $32 \times 8 = 256$ darab bitünk van a teljes üzenetben. Mivel $R = 255$, minden egyes bithez hozzá tudunk rendelni egy univerzálisan egyedi azonosítót.

Algoritmus:

- **Kódolás:** Végigmegyünk az összes $0 \leq i < N$ elemen és azok mind a 8 bitjén ($0 \leq b < 8$). Ha az i-edik elem b-edik bitje **1**-es, akkor elküldünk egy papagájt a következő értékkel:

$$\text{érték} = i \times 8 + b$$

- **Dekódolás:** Inicializálunk egy N méretű nullás tömböt. Minden beérkező $X[j]$ számra kiszámoljuk, melyik bitet jelöli:

$$\text{index} = X[j]/8, \quad \text{bit_pozíció} = X[j] \pmod 8$$

  Az így kapott pozíciójú bitet beállítjuk 1-esre az üzenetünkben.

Papagájok száma (K): Legrosszabb esetben minden bit 1-es, ekkor $8N$ papagájt küldünk. Mivel $8N \leq 10N$, ez a megoldás tökéletesen teljesíti a Subtask 4-et (újabb 47 pont, összesen 81 pont!).

Subtask 5: Globális megközelítés (Elméleti optimum)

Nézzük meg a feladatot globálisan a teljes üzenetre ($\mathtt{N} = 64$, $\mathtt{R} = 255$). A célunk a legmagasabb pontszámot érő $\mathtt{P} \leq 5$ arány, azaz maximum $5 \times 64 = 320$ papagáj használata.

Alapötlet: Szeretnénk kódolni és dekódolni egy 64 elemű, 256-nál kisebb egészekből álló tömböt egy tetszőleges sorrendű, 320 elemű, szintén 256-nál kisebb egészekből álló sorozattal.

A kombinatorikai háttér:

- A 64 elemű eredeti tömb $256^{64} = 2^{512}$ különböző állapotot vehet fel.
- Mivel a papagájok érkezési sorrendje lényegtelen, a megérkező 320 szám egy **monoton növekvő sorozatot** (multihalmazt) alkot.
- Az ilyen sorozatok száma az ismétléses kombináció képletével:

$$\binom{\text{papagáj} + \text{értékkészlet}}{\text{értékkészlet}} = \binom{320 + 256}{256} = \binom{576}{256} \approx 2.34 \times 10^{168}$$

Mivel $\binom{576}{256} > 2^{512}$, a 64 elemű tömbök száma valamivel kevesebb, mint a 320 elemű monoton növekvő sorozatok száma. Így elméletileg létezik egy közvetlen egy-egy értelmű leképezés (bijekció) közöttük.

Blokkosítás

A probléma a globális megközelítéssel: Bár a matematikai bijekció létezik, a megvalósításához **nagyon nagy számok** kezelésére lenne szükség, hiszen 2^{512} állapotot kellene sorszámoznunk.

A praktikus megoldás: Blokkosítás

- Osszuk fel a 64 számot 16 darab, egyenként **4 elemű blokkra**.
- Egy 4 elemű blokk állapota: $256^4 = 2^{32}$. Ez kényelmesen elfér egy standard `long long` változóban, nincs szükség bignum könyvtárra.
- Minden blokknak adunk 16 egyedi vödröt. Ha tartjuk a $\mathtt{P} = 5$ arányt, akkor egy blokkra alapesetben legfeljebb $4 \times 5 = 20$ papagájt használhatunk fel.

Mivel $\binom{20+16}{16} = \binom{36}{16} = 7\,307\,872\,110 > 2^{32}$, a leképezés a 32-bites tartományon belül tökéletesen kódolható.

Finomhangolás (Csonka blokk): Ha `N` nem osztható 4-gyel, az utolsó blokk csonka lesz. A pontpazarlás (padding overhead) elkerülése érdekében itt a maximális papagájszámot a valós mérethez igazítjuk (pl. 1 megmaradt elem esetén elég **3 papagáj**, hiszen $\binom{3+16}{16} = 969 \geq 256$).

Kombinációk sorszámozása (Bijekció)

Hogyan képezzük le a $[0, 2^{32} - 1]$ közötti blokk-értéket a 16 vödörben lévő papagájok darabszámaira? **Kombinatorikus sorszámozással**.

Keressük meg azt az $(x_0, x_1, \ldots, x_{15})$ nemnegatív sorozatot, ahol $\sum x_i \leq 20$, ami a lexikografikos sorrendben a W-edik ezek közül.

Az algoritmus alapelve: Sorban meghatározzuk $x_0, x_1, \ldots$ értékeit. Ha az aktuális vödörbe v darab papagájt teszünk, a maradék vödrökbe még $rem_p - v$ papagájt kell elosztanunk. Az ehhez tartozó kombinációk száma:

$$C = \binom{(rem_p - v) + rem_v - 1}{rem_v - 1}$$

Ha $W \geq C$, akkor levonjuk C-t W-ből, és megpróbálunk eggyel több papagájt tenni ugyanide ($v++$). Ha $W < C$, akkor eldőlt, hogy pontosan v papagáj kerül ide; továbblépünk a következő vödörre (rem_v--, $rem_p -= v$).

Mivel a kombinációk száma belefér a 64 bites egészbe (`long long`), a leképezés oda-vissza gyorsan kiszámolható.

Kártyatrükk (EJOI 2020)

Adott egy standard 52 lapos pakli, a lapok értékei egyedi egészek: `0, 1, ..., 51`. A lapok kezdetben képpel felfelé hevernek az asztalon egy sorban, ismeretlen sorrendben. Két játékosnak kell együttműködnie (titkos kommunikáció nélkül) az alábbiak szerint:

1. fázis: Az Első Játékos

- Odalép az asztalhoz, látja a teljes paklit és a lapok sorrendjét.
- Legfeljebb `S` alkalommal megcserélhet két tetszőleges lapot.
- Ezután távozik, és az összes lapot **képpel lefelé** fordítják (a sorrendjük nem változik).

2. fázis: A Második Játékos

- Odalép az asztalhoz, és megkap egy keresett értéket (`target`).
- Nem lát semmit, de legfeljebb `T` alkalommal megfordíthat egy-egy lapot.
- **Győzelmi feltétel:** Ha a felfordított lapok között szerepel a `target`, a játékosok nyertek. Ha kifogynak a tippekből, veszítenek.

Interfész és korlátok

A feladatot két teljesen különálló programként (modulként) kell megvalósítani, amelyek nem kommunikálhatnak egymással és nem osztoznak globális memórián.

Megvalósítandó függvények:

- `void swapCards(int cards[], int S, int T)`
  Biztosított segédfüggvény: `void doSwap(int i, int j)` (max. `S`-szer hívható).
- `void guessCard(int S, int T, int target)`
  Biztosított segédfüggvény: `int guess(int idx)` (max. `T`-szer hívható, visszaadja az adott indexen lévő lap értékét).

Korlátok: $1 \leq$ `S` ≤ 52, $1 \leq$ `T` ≤ 51, $0 \leq$ `target` < 52.

Részfeladatok:

1. (16 pont): `S` = 52, `T` = 1
2. (20 pont): `S` + `T` = 52
3. (22 pont): `S` = 13, `T` = 27
4. 18 pont): `S` = 1, `T` = 26
5. (24 pont): A nyerő stratégia létezik a megadott `S` és `T` párosra.

Ötlet: A ciklus-követés

Tekintsük a kártyákat egy permutációnak és vegyük a permutáció gráfját, amely az `i` $\rightarrow$ `cards[i]` élekből épül fel. Mivel minden pozíción pontosan egy kártya van és minden érték egyedi, a gráf diszjunkt körökből (ciklusokból) áll.

A Második Játékos stratégiája:

- Elindul a `target` indexről, és megnézi a kártyát.
- Ha a kártya értéke `V`, akkor a következő lépésben a `V`. indexen lévő kártyát fordítja fel.
- Ezt addig ismétli, amíg meg nem találja a keresett értéket.

Következmény: A Második Játékos **mindig** megtalálja a kártyát, a kérdés csak az, hogy hány lépésben. A felhasznált tippek száma pontosan megegyezik annak a ciklusnak a hosszával, amiben a `target` található.

Az Első Játékos célja: Úgy módosítani a paklit legfeljebb `S` cserével, hogy a legnagyobb ciklus hossza is legfeljebb `T` legyen.

Egyszerűbb részfeladatok

Nézzük meg, hogyan működik ez az elv a speciális korlátok mellett:

1. **részfeladat (**$\texttt{S} = 52$**,** $\texttt{T} = 1$**)**
   - $\texttt{T} = 1 \implies$ Minden ciklus hossza maximum 1 lehet. Ez azt jelenti, hogy minden kártyának a saját indexén kell állnia (`cards[i] = i`).
   - Mivel $\texttt{S} = 52$ csere áll rendelkezésre, az Első Játékosnak egyszerűen **sorba kell rendeznie** a paklit (ehhez legfeljebb 51 csere mindig elég). A Második Játékosnak elég 1 tipp: azonnal a `target` indexre bök.

4. **részfeladat (**$\texttt{S} = 1$**,** $\texttt{T} = 26$**)**
   - $\texttt{T} = 26 \implies$ Nem lehet 26-nál hosszabb ciklus a gráfban.
   - Mivel a pakli 52 lapos, **legfeljebb egy darab** olyan ciklus létezhet, aminek a hossza nagyobb, mint 26 (hiszen $2 \times 27 = 54 > 52$).
   - Ha van ilyen óriás-ciklus (hossza $\texttt{L}$), azt az egyetlen megengedett cserénkkel ($\texttt{S} = 1$) pontosan ketté tudjuk vágni egy 26-os és egy $\texttt{L} - 26 \leq 26$ hosszúságú kisebb ciklusra.

Hogyan vágjunk ketté egy ciklust?

Ha egy ciklusban kiválasztunk két tetszőleges indexet (`i` és `j`), és **megcseréljük a két ott lévő kártyát**, azzal a meglévő ciklust pontosan két kisebb, független ciklusra szakítjuk szét.

A pontos vágási mechanizmus:

- Ha egy `L` > `T` hosszúságú ciklusból le akarunk választani egy pontosan `T` hosszúságú darabot:
- Kiválasztunk egy tetszőleges `i` indexet a ciklusból.
- Lépkedünk előre a cikluson pontosan `T` alkalommal, így eljutunk egy `j` indexhez.
- Ha végrehajtjuk a `doSwap(i, j)` műveletet, a ciklus kettéválik:
  1. Egy pontosan `T` hosszúságú ciklusra.
  2. És egy megmaradó, `L` – `T` hosszúságú ciklusra.

A 100 pontos mohó stratégia

A teljes pontszámhoz (Subtask 2, 3, 5) az Első Játékosnak egy optimális, mohó stratégiát kell alkalmaznia a cserék minimalizálására.

Az algoritmus menete:

1. Feltérképezzük a pakli aktuális ciklusait és meghatározzuk a hosszaikat.
2. Keressünk egy olyan ciklust, amelynek hossza $\texttt{L} > \texttt{T}$.
3. Az előző dián bemutatott módszerrel **levágunk belőle egy** $\texttt{T}$ **hosszúságú darabot** 1 csere árán. A ciklus megmaradt része $\texttt{L} - \texttt{T}$ hosszúságú lesz.
4. Ezt addig ismételjük, amíg a megmaradt rész is $\leq \texttt{T}$ nem lesz, vagy amíg van $\texttt{T}$-nél nagyobb ciklusunk.

Miért működik? Mivel a feladat szövege garantálja, hogy a megadott $\texttt{S}$ és $\texttt{T}$ paraméterek mellett létezik nyerő stratégia, ez a mohó darabolás mindig a lehető legkevesebb cserét fogja elhasználni, így biztosan nem lépjük túl az $\texttt{S}$ korlátot.

Stations (IOI 2020)

A Singapore's Internet Backbone (SIB) hálózat `n` állomásból áll, amelyek egy fát (tree) alkotnak. Az állomások kezdeti indexei 0-tól `n` – 1-ig terjednek.

A feladatunk egy kétlépcsős útvonalválasztási séma megvalósítása:

- **Címkézés (Ali):** Egyedi egész címkéket kell hozzárendelnünk minden állomáshoz 0 és `k` között. Célunk a használt maximális címke (`m`) minimalizálása.
- **Útvonalválasztás (Brenda):** A csomagot továbbító függvény csak a jelenlegi állomás címkéjét (`s`), a célállomás címkéjét (`t`) és a közvetlen szomszédok rendezett címkéit (`c`) látja. Vissza kell adnia a következő lépés címkéjét.

Szigorú korlát: Nincs lehetőség globális vagy statikus memória használatára a két különálló programfutás között.

Interfész és részfeladatok

Megvalósítandó függvények:

- `int[] label(int n, int k, int[] u, int[] v)`
- `int find_next_station(int s, int t, int[] c)`

Globális korlátok: $2 \leq \mathtt{n} \leq 1000$.

Részfeladatok és a pontozás a max címke ($\mathtt{m}$) függvényében:

1. **Subtask 1 (5 pont):** A hálózat egy vonal, $\mathtt{k} = 1000$.
2. **Subtask 2 (8 pont):** Bináris fa struktúra, $\mathtt{k} = 1000$.
3. **Subtask 3 (16 pont):** Legfeljebb egy csúcs foka > 2, $\mathtt{k} = 1\,000\,000$.
4. **Subtask 4 (10 pont):** Nagyon kis fa ($\mathtt{n} \leq 8$), $\mathtt{k} = 10^9$.
5. **Subtask 5 (61 pont):** Általános fa, $\mathtt{k} = 10^9$.
   - $\mathtt{m} \leq 1000 \implies$ **Teljes 61 pont** (Összesen 100 pont).
   - $1000 < \mathtt{m} < 2000 \implies$ Fixen 50 pont.
   - $2000 \leq \mathtt{m} < 10^9 \implies$ Részpontszám logaritmikus képlet alapján.

A stratégia alapja: ős-leszármazott viszony

Gyökereztessük le a fát a legkisebb címkéjű csúcsban. Ha a címkék alapján Brenda képes eldönteni két tetszőleges csúcsról, hogy melyik melyiknek az őse, a routing lokálisan megoldható:

Brenda döntési algoritmusa a forrás (`s`) csúcsnál:

1. Ellenőrzi a szomszédok (`c`) között, hogy van-e olyan, ami **őse** `s`**-nek**. Ha van, az a szülő (`P`). (Legfeljebb 1 ilyen van).
2. A többi szomszéd mind `s` gyereke. Megnézi, hogy valamelyik gyerek **őse-e a célnak (`t`)**. Ha igen, a csomagot ennek a gyereknek küldi.
3. Ha egyik gyerek sem őse `t`-nek, akkor a célcsúcs vagy `s` felett van a fában, vagy egy teljesen másik részfában. Ekkor a csomagot a szülőnek (`P`) kell adni.

Első megközelítés: Euler-bejárás (65 pont)

Futtassunk egy mélységi bejárást (DFS) a gyökérből, és mentsük el a csúcsokba lépés (`in[v]`) és kilépés (`out[v]`) időpontjait.

Egy v_1 csúcs pontosan akkor őse v_2-nek, ha $in[v_1] < in[v_2] < out[v_1]$.

Egyszerű kódolás (Subtask 5 részpont):

- Ali rendelje hozzá a `v` csúcshoz a következő címkét: $L = in[v] \times 1000 + out[v]$.
- Brenda ebből könnyen visszafejtheti a határokat: $in[v] = L/1000$ és $out[v] = L\%1000$.
- A maximális címke így 999 9999 körül alakul, ami az első subtaskok külön kezelésével kb. **65 pontot** ér.

Trükk a 89 pontért: mélység paritás

Vegyük észre, hogy egy csúcs és szomszédai szorosan összefüggő `in` és `out` értékekkel bírnak. Ha a gyerekeket `in` szerint rendezzük, akkor $\texttt{out}[c_1] = \texttt{in}[c_2] - 1$, valamint $\texttt{in}[s] = \texttt{in}[c_1] - 1$. Felesleges mindkét értéket eltárolni csúcsonként!

Az eltolt címkézés ötlete (max label $\leq 2n$):

- Ha a `v` csúcs távolsága a gyökértől **páros**: a címkéje legyen `in[v]`.
- Ha a távolsága **páratlan**: a címkéje legyen `out[v]`.

Hogyan tudja Brenda rekonstruálni az adatokat?

- Ha `s` címkéje kisebb, mint az összes szomszédjáé, akkor `s` páros szinten van (a saját címkéje `in[s]`). A szomszédai mind páratlan szinten vannak, így az ő címkéik az `out` értékeik.
- Ellenkező esetben `s` páratlan szinten van (címkéje `out[s]`), a szomszédoké pedig az `in` értékük.

A hiányzó intervallum-határokat Brenda a szomszédok rendezett címkéinek egymáshoz viszonyított értékéből le tudja vezetni. Ez a megoldás már **89 pontot** ad.

A 100 pontos megoldás: címketömörítés

A 89 pontos megoldásban használt címkék abszolút értéke nem lényeges, Brenda algoritmusa kizárólag a címkék egymáshoz képesti **sorrendjét (relációját)** használja fel a döntésekhez.

A végső optimalizáció (Ali):

1. Számítsuk ki minden csúcsra a 89 pontos megoldás által javasolt ideiglenes címkét (`in[v]` vagy `out[v]` paritástól függően).
2. Gyűjtsük össze ezt az `n` darab egyedi számot, és határozzuk meg a növekvő sorrendben vett rangjukat.
3. Minden csúcs végleges címkéje legyen a saját ideiglenes címkéjének a rangja (0 és $n-1$ között).

Eredmény: Mivel a rangsorolás tökéletesen megőrzi a címkék egymás közti egyenlőtlenségeit, Brenda logikája érintetlenül működik. A maximális címke garantáltan $m = n - 1 \leq 1000$ lesz, ami megadja a **maximális 100 pontot**.

Two Transportations (JOI 2019)

Adott $\mathtt{N}$ város egy országban (0-tól $\mathtt{N}-1$-ig számozva). A cél a legrövidebb út megtalálása a 0-s városból az összes többibe.

A csavar: Megosztott információ

- **Azer** csak a vasútvonalakat ismeri.
- **Baijan** csak a buszvonalakat ismeri.
- A két programnak együtt kell működnie, de **csak 0 és 1 bitek küldésével** kommunikálhatnak.

Szigorú korlátok:

- $\mathtt{N} \leq 2000$.
- A két játékos összesen legfeljebb **58 000 bitet** küldhet egymásnak.
- Egy él költsége (vonat vagy busz) maximum 500 lehet.

Az alapötlet: Elosztott Dijkstra

A probléma egy egyforrású legrövidebb út keresés, amit ezen a gráfon a Dijkstra-algoritmus old meg hatékonyan.

A Dijkstra induktív tulajdonsága:

- Ha már ismerjük a k darab legközelebbi csúcs végleges távolságát, tudjuk, hogy a $(k+1)$-edik legközelebbi csúcs **biztosan szomszédos** az eddig megtalált csúcsok valamelyikével.

A közös munka menete:

- Mindkét játékos lokálisan vezeti a saját rész-Dijkstra algoritmusát.
- Minden körben mindketten megnézik, hogy a saját gráfjuk (vonat vagy busz) alapján melyik csúcs a legígéretesebb következő jelölt.
- **Kommunikáció:** Összehasonlítják a két jelöltet. Amelyiké kisebb, az a globálisan is érvényes $(k+1)$-edik csúcs.
- Ezt szinkronizálják (távolság + csúcs azonosító), és mindketten relaxálják a saját lokális éleiket ezen az új csúcson keresztül.

A naiv kommunikációs protokoll (53N bit)

Hogyan szinkronizáljuk a jelölteket? Tegyük fel, hogy A kezd:

1. A elküldi a legjobb jelöltjének távolságát $\rightarrow$ **20 bit**.
2. B összehasonlítja a saját jelöltjével (ehhez nem kell bit).
3. **Ha B jelöltje jobb:** B üzen 1 bitet (jelezve, hogy nyert), majd elküldi a saját távolságát (**20 bit**) és a csúcs sorszámát (**11 bit**, mivel $N \leq 2000$).
4. **Ha A jelöltje jobb:** B üzen 1 bitet (jelezve, hogy A nyert), majd megkéri A-t, hogy küldje el a csúcs sorszámát (**11 bit**).
5. "Ébresztő" bit: 1 bittel jelzünk A-nak, hogy kezdheti előlről.

Összköltség: Körönként $20 + 1 + 20 + 11 + 1 = 53$ bit.
Mivel minden csúcsot meg kell találni, ez $53 \times 2000 = 106\,000$ bitet jelent, ami sajnos durván túllépi az 58 000-es limitet.

A kulcs-optimalizáció: Delta kódolás (31N bit)

Vegyük észre a feladat legfontosabb korlátját: egyetlen él (vonal) költsége sem haladja meg az 500-at!

Következmény: Az újonnan megtalált $(k+1)$-edik legközelebbi csúcs távolsága sosem lehet nagyobb, mint az eddig felfedezett maximum távolság (`max_dist`) +500.

Delta kódolás:

- Teljes 20-bites távolságok helyett elegendő a **távolság-növekményt** ($\Delta = \texttt{dist} - \texttt{max_dist}$) elküldeni!
- Mivel $0 \leq \Delta \leq 500$, ez a szám kényelmesen elfér **9 biten** (ahol az 511-es érték kódolhatja a "végtelent", ha nincs több elérhető csúcs).

Új összköltség: 9 (távolság) +1 (döntés) +9 (másik távolság) +11 (csúcs) +1 (ébresztő) = 31 bit.
$31 \times 2000 = 62\,000$ bit. Ez már majdnem befér!

Konstans optimalizáció a 100 pontért (29N bit)

Faragjunk még le 2 bitet körönként a felesleges "beszélgetésből"!

Implicit jelzések:

- Nem kell külön meta-bitet (1 bit) küldeni arról, hogy ki nyert.
- Ha A átküldi a deltáját (9 bit), és B észreveszi, hogy az ő jelöltje a jobb, B egyből elküldi a saját deltáját (9 bit) és a csúcsot (11 bit).
- Ha B tudja, hogy A a nyertes, B egy "végtelen" (511) jelzést küld 9 biten. A ebből rájön, hogy ő nyert, és elküldi a 11 bites csúcs-azonosítót.

Ébresztő bitek elhagyása: Nem kell bitet pazarolni az újraindításra. Mindkét játékos lokálisan vezeti, hol tartanak, és A automatikusan indítja a következő kört, ha a relaxáció befejeződött.

Végeredmény: Ezzel a csonkítással lejutunk **29 bit / csúcs** költségre. $29 \times 2000 = 58\,000$ bit, ami pont tökéletes!

Továbbfejlesztés: Randomizáció (25.5N bit)

Randomizációval tovább csökkenthető az *átlagos* bitszám.

A stratégia:

- A két program generáljon egy azonos, szinkronizált pszeudo-véletlenszám sorozatot (közös hardkódolt seed alapján).
- Minden körben ez dönti el ("pénzfeldobás"), hogy `A` vagy `B` kezdje a 9 bites küldést.
- Ha az elsőként üzenő fél deltája kisebb, mint a másodiké, a másodiknak már nem is kell visszaküldenie a saját 9 bitjét, hiszen egyből egyértelmű a nyertes!

Várható bitszám: Körülbelül az esetek felében megspóroljuk a második 9 bites üzenetet.

$$9 \text{ (első)} + 1 \text{ (válasz)} + 9 \times 0.5 \text{ (második esélye)} \approx 14.5 \text{ bit a távolságra}$$

Összesen $14.5 + 11$ (csúcs) = **25.5 bit** körönként! Ezzel a módszerrel a protokoll robusztus marad, és jócskán a limit alatt teljesít.

Message (IOI 2024)

Aisha egy $\mathtt{S}$ hosszúságú ($\mathtt{S} \leq 1024$) bináris üzenetet szeretne küldeni Basmának 31 bites csomagok formájában.

A probléma (Kleopátra zavarása):

- A hálózatot Kleopátra figyeli, aki minden csomagban pontosan 15 bitet kénye-kedve szerint módosíthat.
- Ezek a módosítható bitek az egész kommunikáció alatt ugyanazok (egy $\mathtt{C}$ maszk).
- **Az aszimmetria:** Aisha pontosan tudja a $\mathtt{C}$ maszkot (melyik 15 bit sérülékeny és melyik 16 biztonságos), de Basma nem tudja!

A cél: Olyan stratégiát találni, amivel Basma hibátlanul vissza tudja állítani az eredeti üzenetet a szennyezett csomagokból.

Részfeladatok és pontozás: A pontszám a felhasznált csomagok maximális számától ($\mathtt{Q}$) függ. $\mathtt{Q} \leq 66$: $\mathtt{X} = 1$, $\mathtt{Q} > 100$: $\mathtt{X} = 0$, $66 < \mathtt{Q} \leq 100$: $\mathtt{X} = 0.95^{\mathtt{Q}-66}$ (exponenciálisan csökken).

- 1. részfeladat: $10 \cdot \mathtt{X}$ pont ($\mathtt{S} \leq 64$)
- 2. részfeladat: $90 \cdot \mathtt{X}$ pont (általános $\mathtt{S}$)

Subtask 1: Többségi szavazás (10 pont)

Ha az üzenet rövid ($S \leq 64$), megtehetjük, hogy csomagonként csak 1 bit hasznos információt küldünk.

A stratégia (Ismétléses kódolás):

- **Aisha:** Ha a küldendő bit `0`, egy csupa nullából (`000...000`) álló csomagot küld. Ha a bit `1`, akkor egy csupa egyesből (`111...111`) álló csomagot küld.
- **Basma:** Megszámolja a kapott csomagban a nullákat és az egyeseket. Amelyikből több van (a többség), az volt az eredeti bit.

Miért működik? Mivel egy csomag 31 bites, és Kleopátra legfeljebb 15 bitet tud megváltoztatni, a maradék 16 bit garantáltan érintetlen marad. Így a helyes bit mindig többségben lesz (legalább $16 > 15$ arányban). A kódoláselméletben ezt *repetition code*-nak hívják.

A Q = 96 csomagos megoldás (Alapötlet)

Ha S akár 1024 is lehet, a csomagonként 1 bit túl lassú. Ehelyett Aisha megtaníthatja Basmának, hogy melyik az a 16 biztonságos bit.

A maszk átküldése és a Padding:

1. **A maszk küldése:** Aisha az első 31 csomagot arra használja, hogy a Subtask 1 többségi szavazásával átküldje a C maszk 31 bitjét.
2. **Padding:** Mivel Basma nem ismeri előre S-t, Aisha kipótolja az üzenetet pontosan 1025 bitre: az üzenet elé nullákat, majd egy 1-est tesz jelzőbitként.
3. **Üzenetküldés:** Miután Basma már ismeri a 16 biztonságos pozíciót, Aisha csomagonként 16 bitet tud küldeni ezeken a helyeken.

Összköltség: 31 (maszk) + $\lceil 1025/16 \rceil$ (üzenet) = 31 + 65 = 96 csomag.

Optimalizációk a maszkon ($Q = 85$ és $Q = 71$)

Miért pazarolnánk el 31 csomagot a maszkra? Csak azért küldtünk 1 bitet csomagonként, mert Basma még nem ismert biztonságos biteket.

Az iteratív gyorsítás ($Q = 85$): Amint Basma az első pár csomagból azonosított k darab biztonságos bitet, Aisha a következő csomagokban már k darab maszk-bitet tud átküldeni egyszerre ezeken a pozíciókon. A legrosszabb esetben (ha az első 15 bit mind sérülékeny) ez a fázis 31 helyett mindössze 20 csomag alatt lezajlik ($20 + 65 = 85$).

Egyetlen bit azonosítása ($Q = 71$): Még jobb, ha Aisha az első 5 csomagban (binárisan kódolva, a többségi szavazást használva) csak **egyetlen egy** biztonságos bit indexét küldi át. Utána ezt az 1 biztonságos bitet használja a maradék 15 index átküldésére, míg a többi felfedezett biten már azonnal áramolhat a hasznos üzenet. Ezzel a módszerrel elérhető a $Q = 71$.

A 100 pontos megoldás felé: gráfos megközelítés

Felejtsük el az eddigieket, és nézzük a problémát teljesen új szemszögből. Legyen az indexe a 16 darab biztonságos (nem módosítható) bitnek növekvő sorrendben $a_1, a_2, \ldots, a_{16}$.

A láncolat elküldése ($Q = 70$):

- Aisha az a_i-edik bit pozícióján az első 5 csomagon keresztül elküldi a **következő** biztonságos bit (a_{i+1}) indexét binárisan kódolva.
- Basma az első 5 csomag alapján minden egyes $0 \leq i \leq 30$ pozícióra kap egy 5-bites értéket (b_i).
- Basma felépít egy 31 csúcsból álló irányított gráfot, ahol minden i csúcsból egyetlen él mutat a b_i csúcsba.
- Kleopátra akármit is csinál a többi 15 bittel, a gráfban **pontosan egy darab 16 hosszúságú kör** fog létrejönni, ami egyértelműen felfedi a biztonságos biteket!
- Mivel 5 csomag kell a szerkezethez, és 65 a maradék üzenethez, ez a módszer $Q = 70$ csomagot igényel.

Egy apró finomítás ($Q = 69$): Ha az abszolút indexek helyett az $(a_{i+1} - a_i) \pmod{31}$ távolságot küldjük, az maximum 16 lehet, így 5 helyett 4 bit (csomag) is elég a struktúra elküldésére.

A 100 pontos megoldás: one-hot kódolás ($Q = 66$)

A teljes megoldáshoz a bináris kódolást **one-hot kódolásra** cseréljük.

A one-hot kódolás működése:

- Minden V értéket $V - 1$ darab nullával, majd a legvégén egyetlen 1-essel kódolunk.
- Például a $1 \to 1$, a $2 \to 01$, a $3 \to 001$, és az $5 \to 00001$.
- Ezt a kódolt bitsorozatot Aisha folyamatosan belefolyatja a biztonságos bitek láncolatába a csomagokon keresztül.

A korlátok pontos kiaknázása:

- A One-hot kódolás bitigénye pontosan megegyezik az ábrázolt értékek összegével.
- Mivel egy 31 hosszú gyűrűn érünk körbe, a távolságok összege garantáltan: $\sum((a_{i+1} - a_i) \pmod{31}) = 31$.
- Tehát pontosan 31 biztonságos bitet emészt fel a gráf felépítése a folyamat során.
- Ha összesen 66 csomagot küldünk, akkor $66 \times 16 = 1056$ biztonságos bit áll rendelkezésre.
- $1056 - 31 =$ **1025** szabad bit marad.
- Ez a szám pedig **pontosan** megegyezik a maximális üzenet hosszával.

Különkiadás: Prison (IOI 2022)

Megjegyzés: Ez technikailag egy stratégia-tervező feladat (egy 2D tömböt kell visszaadni), de az elve lényegében egy aszinkron állapotgép típusú interaktív feladat.

A probléma:

- 500 rab megy be egyesével egy szobába, ahol két zsák van (`A` és `B`), mindkettőben $1 \dots \texttt{N}$ érme ($\texttt{N} \leq 5000$, $\texttt{A} \neq \texttt{B}$). A cél kitalálni, melyikben van kevesebb.
- A szobában van egy tábla, amin egyetlen szám áll: $0 \dots \texttt{x}$.
- A belépő rab elolvassa a táblát, **csak az egyik** zsákba nézhet bele, majd vagy megmondja a megoldást, vagy felír egy új számot a táblára, és kimegy.

A cél és a pontozás: Minimalizálni a táblára írható legnagyobb számot (`x`). A 100 ponthoz $\texttt{x} \leq 20$ elérése szükséges $\texttt{N} = 5000$ mellett.

A koncepció: Alternáló intervallum-szűkítés

Mivel egy rab csak az egyik zsákot látja, a táblának kell hordoznia a másik zsákról szerzett eddigi információt. A keresési teret egy `[L, R]` intervallummal írjuk le.

Az ingyenes végpontok trükkje: Tegyük fel, tudjuk, hogy mindkét zsák érméinek száma az `[L, R]` intervallumba esik.

- A rab belenéz az `A` zsákba.
- Ha $A = L$, akkor azonnal tudja, hogy $A < B$ (hiszen $B \geq L$ és $A \neq B$).
- Ha $A = R$, akkor azonnal tudja, hogy $A > B$.
- **Következmény:** Pusztán az intervallum szélének ellenőrzésével 2 lehetséges értéket azonnal kizártunk extra üzenet nélkül!

A felosztás: Ha az `A` zsák tartalma a belső $[L + 1, R - 1]$ részbe esik, ezt a maradék tartományt felosztjuk `k` darab kisebb al-intervallumra. A rab felírja a táblára azt a `v` állapotot, ami jelzi, hogy *"Az `A` zsák a `k`-adik al-intervallumban van"*. A következő rab már a `B` zsákot fogja kinyitni, és ebben a szűkebb intervallumban ismétli a folyamatot.

Állapotok kódolása és a rekurzió

Mit is kódol egy $\mathtt{v}$ szám a táblán? Két dolgot egyszerre:

1. **Melyik zsákot kell kinyitni:** (pl. 0. szint $\rightarrow \mathtt{A}$, 1. szint $\rightarrow \mathtt{B}$, 2. szint $\rightarrow \mathtt{A}$).
2. **Melyik al-intervallumba** esett az előzőleg vizsgált zsák.

Mivel a szintek szigorúan váltakoznak, a $0 \ldots 20$ állapotokat **diszjunkt halmazokra** (szintekre) oszthatjuk. A 0 a kezdőállapot. Marad 20 értékünk, amit fel kell osztanunk.

A rekurzió képlete: Ha egy adott szinten $\mathtt{k}$ darab állapotot (al-intervallumot) engedélyezünk, és a következő szint egy $\mathtt{S_{next}}$ méretű intervallumot tud feldolgozni, akkor a jelenlegi szint maximális kapacitása:

$$\mathtt{S_{prev}} = \mathtt{k} \times \mathtt{S_{next}} + 2$$

(A +2 a két ingyenes végpont, amit állapotpazarlás nélkül tudunk ellenőrizni).

A 100 pontos matematikai csoda ($x = 20$)

Osszuk el a rendelkezésre álló 20 állapotot 7 szint között a következőképpen: **3, 3, 3, 3, 3, 3, 2** (Ezek összege pontosan 20).

Számoljuk ki lentről felfelé (a levelektől a gyökérig), mekkora S intervallumot képes ez a struktúra feldolgozni:

- **7. szint (levél):** Nincs több állapot. $S_7 = \mathbf{2}$. (Csak a két végpont ellenőrizhető).
- **6. szint (2 állapot):** $S_6 = 2 \times S_7 + 2 = 2 \times 2 + 2 = \mathbf{6}$.
- **5. szint (3 állapot):** $S_5 = 3 \times S_6 + 2 = 3 \times 6 + 2 = \mathbf{20}$.
- **4. szint (3 állapot):** $S_4 = 3 \times S_5 + 2 = 3 \times 20 + 2 = \mathbf{62}$.
- **3. szint (3 állapot):** $S_3 = 3 \times S_4 + 2 = 3 \times 62 + 2 = \mathbf{188}$.
- **2. szint (3 állapot):** $S_2 = 3 \times S_3 + 2 = 3 \times 188 + 2 = \mathbf{566}$.
- **1. szint (3 állapot):** $S_1 = 3 \times S_2 + 2 = 3 \times 566 + 2 = \mathbf{1700}$.
- **0. szint (kezdő állapot):** $S_0 = 3 \times S_1 + 2 = 3 \times 1700 + 2 = \mathbf{5102}$.

Konklúzió: Mivel $5102 \geq 5000$, pontosan 20 állapot elegendő a teljes $N = 5000$ tartomány lefedésére. A megoldás egy kevert alapú (vegyesen 3-as és 2-es) intervallum-felező/harmadoló fa, ami optimálisan kihasználja a végpontok adta ingyenes információt.

Kitekintés: Transfer of Duty (Universal Cup)

Egy laboratóriumban 10^6 darab kapcsoló található, amelyek kezdetben mind kikapcsolt (off) állapotban vannak. A nap folyamán a dolgozók különböző kapcsolókat billentenek át (on $\leftrightarrow$ off).

A feladat: Minden egyes kapcsolás után azonnal meg kell mondanunk a rendszer aktuális állapotát:

- **0**: Minden eszköz ki van kapcsolva.
- **ID**: Pontosan egy eszköz van bekapcsolva (ilyenkor meg kell adni a számát 1 és 10^6 között).
- **-1**: Kettő vagy több eszköz van bekapcsolva.

A csavar (Suspend and Resume): A feladatot két fázisban kell megoldani, és a programunk kétszer fog lefutni a szerveren:

1. **Első futás:** Végrehajt legfeljebb 100 000 kapcsolást, majd a műszak végén hagy egy maximum **1000 karakteres** (ASCII kódok: 32-126) üzenetet a váltótársának.
2. **Második futás:** A program újraindul, beolvassa az előzőleg hagyott rövid üzenetet, majd folytatja a munkát újabb legfeljebb 100 000 kapcsolással.

A kihívás és a megoldás: XOR hashelés

A probléma: Hogyan kódoljuk 10^6 kapcsoló potenciális állapotát egy mindössze 1000 karakteres üzenetbe úgy, hogy a második futásnál is azonnal tudjuk, mi van bekapcsolva?

A trükk:

- Rendeljünk minden kapcsolóhoz egy egyedi, 64-bites véletlenszámot, az `i`-edikre: `hash(i)`.
- **Fontos implementációs részlet:** A véletlenszám-generátort (pl. C++-ban a `std::mt19937_64`-et) egy **fix seeddel** kell inicializálni a program elején. Így garantálható, hogy az Első és a Második futás is *pontosan ugyanazokat* a hash-értékeket generálja le a kapcsolókhoz, anélkül, hogy ezeket át kéne adni az üzenetben!

Megjegyzés: Bár a klasszikus hashelés szigorú értelemben véve az IOI syllabusán kívül esik, a véletlenszámokon alapuló technikák ismerete a versenyprogramozásban hatalmas előnyt jelenthet.

Állapotgép és az üzenet

A teljes rendszer állapotát egyetlen 64-bites egész számban (C) fogjuk tárolni, amely kezdetben 0.

Állapotfrissítés: Amikor az x-edik kapcsolót átbillentik, egyszerűen alkalmazzuk a bitenkénti kizáró vagy (XOR) műveletet:

$$C = C \oplus \texttt{hash}(x)$$

Mivel a XOR művelet önmaga inverze ($A \oplus A = 0$), egy bekapcsolás hozzáadja a hasht, egy kikapcsolás pedig pontosan kiejti azt.

Az állapot dekódolása:

- Ha $C = 0 \implies$ **Minden ki van kapcsolva** (Kimenet: 0).
- Ha C megtalálható az előre legenerált hash-ek között (pl. egy map-ben vagy egy rendezett tömbben bináris kereséssel) $\implies$ **Pontosan az a kapcsoló van bekapcsolva** (Kimenet: a megtalált ID).
- Minden más esetben $\implies$ **Több eszköz van bekapcsolva** (Kimenet: -1).

Az átadandó üzenet: A műszak végén az üzenet pusztán a C változó 64-bites aktuális értéke lesz (szövegesen formázva legfeljebb ~ 20 karakter). Ez $O(1)$ extra memóriát jelent, és bőven belefér az 1000 karakteres korlátba.

Matematikai biztonság (ütközés esélye)

A randomizált algoritmusoknál mindig felmerül a kérdés: mi van, ha két különböző állapot véletlenül ugyanazt a hash-értéket adja ki?

A valószínűségi becslés:

- 64 bites véletlenszámok esetén a lehetséges értékek száma kb. 10^{18}.
- A speciális állapotok, amiket fel kell ismernünk, az a 0 állapot és a 10^6 darab egyedi `hash(i)` érték.
- Annak az esélye, hogy egy több kapcsolóból álló, véletlenszerű XOR összeg "véletlenül" pont rálép a 10^6 speciális érték valamelyikére egy lépésben:

$$\mathtt{P}(\text{ütközés}) \approx \frac{10^6}{10^{18}} = 10^{-12}$$

Globális hibavalószínűség: A két futás alatt összesen legfeljebb $\mathtt{S} = 2 \cdot 10^5$ lépés történik. Annak az esélye, hogy a teljes folyamat során **legalább egy** lépésben ütközés lép fel, a Boole-egyenlőtlenség alapján felülről becsülhető:

$$\mathtt{P}(\text{hiba}) \leq \mathtt{S} \cdot \mathtt{p} = (2 \cdot 10^5) \cdot 10^{-12} = 2 \cdot 10^{-7}$$

Ez az apró valószínűség ($1 : 5\,000\,000$) azt jelenti, hogy a 64 bites számokkal hashelés megbízható a versenykörülmények között. Nagyon elegáns és tanulságos algoritmus.

Összefoglalás: Kétfázisú feladatok típusai

Az ilyen jellegű (szűkített kommunikációs vagy aszinkron) interaktív feladatok négy fő kategóriába sorolhatók:

1. **Kódoló-Dekódoló**
   - **Cél:** Egy nagyméretű adatstruktúra vagy üzenet átküldése nagyon szűkös vagy zajos csatornán.
   - **Példa:** IOI 2011 Parrots, IOI 2024 Message.
2. **Gráf/Állapot előkészítés (Adatszerkezet építés)**
   - **Cél:** Egy ágens látja a teljes gráfot/adatot, és felcímkézi azt. A második ágens csak lokális részleteket lát, és ezen adatok alapján kell navigálnia.
   - **Példa:** IOI 2020 Stations, EJOI 2020 Cards.
3. **Elosztott feldolgozás (Közös tudás szinkronizációja)**
   - **Cél:** Két ágens az adat felét-felét látja. Párhuzamosan futva kell elérni egy célt, korlátolt információmegosztási lehetőséggel.
   - **Példa:** JOI 2019 Two Transportations (Elosztott Dijkstra).
4. **Aszinkron állapotgép (Üzenőfalas kommunikáció)**
   - **Cél:** Az ágensek felváltva lépnek, és csak egy közös, nagyon kis kapacitású globális memóriát (egy "táblát") írhatnak/olvashatnak.
   - **Példa:** IOI 2022 Prison (technikailag nem interaktív, de lehetne).

Általános megoldási technikák

1. **Kombinatorikus sorszámozás és bijekciók (állapottér-minimalizálás)**
   - **Elv:** Ha a továbbítandó komplex struktúrák (pl. fák, halmazok) száma jóval kisebb, mint a nyers bináris ábrázolásuk, a struktúrákat egyértelműen leképezzük egy-egy sorszámra, és csak ezt a kompakt indexet továbbítjuk.
   - **Alkalmazás:** Lexikografikus indexelés, kombinációk sorszámozása.

2. **Információtömörítés és bázisváltás (linearizálás)**
   - **Elv:** Több független, kis értéktartományú változót (pl. logikai értékek, kis egészek) egyetlen nagyobb egész számmá pakolunk össze, teljesen kiküszöbölve a csatorna kihasználatlan bitjeit.
   - **Alkalmazás:** Bitmaszkolás bitenkénti eltolással (`«`), vagy vegyes alapú számrendszerek használata több különböző állapot kódolására egyetlen értékben (`A` × `B` + `C` formában).

3. **Hibajavító kódolás és vezérelt redundancia**
   - **Elv:** Zajos vagy az ellenfél (adversary) által szándékosan módosított csatornák esetén az információt szisztematikus matematikai redundanciával egészítjük ki, lehetővé téve a hibák automatikus javítását plusz kommunikáció nélkül.
   - **Alkalmazás:** *Ismétléses kódolás* (Repetition Code) és többségi szavazás az egyszerűbb részfeladatokhoz (pl. egy bitet 31-szer elküldeni csomagonként), vagy fejlettebb szinten paritásbitek és Hamming-kódok.

4. **Strukturális invariánsok és topológiai absztrakció**
   - **Elv:** A bonyolult relációk vagy hálózatok globális leírása helyett olyan minimális, lokális jellemző értékeket kódolunk, amelyek tömörítve is megőrzik a probléma szempontjából kritikus topológiai tulajdonságokat.
   - **Alkalmazás:** *Gráfok lapítása Euler-bejárással* (IOI Stations) – a teljes fa-koordináták helyett csak a bejárási időintervallumokat (`in`, `out`) kódoljuk, ami tökéletesen megőrzi az ős-leszármazott relációt.

5. **Kontextus-alapú és inkrementális adatábrázolás**
   - **Elv:** Abszolút adatok és teljes struktúrák átvitele helyett csak egy már ismert bázisállapothoz képesti változást (különbséget), vagy az elemek egymáshoz viszonyított relatív távolságát továbbítjuk.
   - **Alkalmazás:** *Delta-kódolás* (JOI Two Transportations) – monoton növekvő sorozatok vagy távolságok esetén a globális értékek helyett csak a szomszédos elemek különbségét küldjük.

6. **Véletlenítés (közös random seed):**
   - **Elv:** Ha a két ágens ugyanabból a fix seedből indít egy determinisztikus véletlenszám-generátort, kommunikáció nélkül is képesek közös, "véletlenszerű" döntéseket hozni.
   - **Alkalmazás:** Például a JOI Two Transportations feladatban ki kezdjen a kommunikációban, elkerülve a redundanciát, illetve az Universal Cup Transfer of Duty feladatban pszeudo-véletlen hash értékek generálása.

Általános megoldási technikák

7. **Információ-aszimmetria feloldása (A „tanító” modell)**
   - **Elv:** Amikor az egyik ágens teljesen ismeri a környezetet (pl. a tiltott maszkot vagy a gráf éleit), míg a másik „vak”, a kommunikáció első fázisát kizárólag a kontextus és a környezeti szabályok megtanítására fordítjuk. Miután a vak ágens „látóvá” válik, a második fázisban már maximális hatékonysággal és sávszélességgel küldhető a hasznos üzenet.
   - **Alkalmazás:** *A gráfalapú maszk-átvitel* (IOI Message) – Aisha az első néhány csomagot arra használja, hogy a biztonságos bitek láncolatát elküldje. Basma ebből felépíti a gráfot, azonosítja a 16 biztonságos pozíciót, így a következő fázisban már zökkenőmentesen áramolhat a hasznos üzenet.

8. **Kényszer-vezérelt dinamikus kódolás (Elastic Encoding)**
   - **Elv:** A kódolási sémát (a bitek kiosztását) úgy alakítjuk ki, hogy az ábrázolás hossza vagy súlya rugalmasan igazodjon a feladat globális matematikai kényszereihez és határaihoz.
   - **Alkalmazás:** *One-hot kódolás* (IOI Message) – a bitek pozíciójával kódolunk értékeket ott, ahol az értékek (távolságok) összege egy szigorúan kötött felső korláttal rendelkezik.

Feladatok

IOI 2019 Practice – Data Transfer

https://oj.uz/problem/view/IOI19_transfer

Araz egy N bites bináris sorozatot szeretne elküldeni Babának. Az adatátvitel során zaj léphet fel, ezért Araz K extra bitet csatol az adathoz, hogy Baba vissza tudja állítani az eredeti üzenetet.

Szabályok:

- A továbbított összesen $N + K$ bitből legfeljebb egyetlen bit változhat meg (sérülhet) az átvitel során.
- A csatolt bitek száma (K) nem haladhatja meg a 2N-et.
- Két külön eljárást kell megírni: az egyik Araz (kódolás), a másik Baba (dekódolás és hibajavítás) logikáját valósítja meg.

Feladat: Határozzuk meg az elküldendő plusz biteket úgy, hogy Baba mindig helyre tudja állítani az eredeti N bites üzenetet, miközben K értékét (a csatolt bitek maximális számát) a lehető legkisebbre csökkentjük.

Korlátok:

- $N \in \{63, 255\}$
- $1 \leq T \leq 200\,000$ (tesztesetek száma)

JOI 2020 – Stray Cat

https://oj.uz/problem/view/JOI20_stray

Anthony (a hangya) a 0-s városban él. Barátja, Catherine (a macska) eltévedt egy másik városban, és nem ismeri az úthálózatot. Anthony előre megjelöli az utakat, hogy Catherine hazataláljon.

Szabályok:

- A városok közötti úthálózat összefüggő, N várost és M utat tartalmaz.
- Anthony A féle jelölést tehet az utakra.
- Catherine minden lépésben csak azt látja, hogy a jelenlegi városából hány darab indul ki az egyes jelölésű utakból (kivéve azt az utat, amelyiken épp megérkezett).
- A látottak alapján Catherine kiválasztja, hogy melyik jelölésű úton megy tovább.

Feladat: Tervezzük meg Anthony jelölési stratégiáját és Catherine navigációs logikáját úgy, hogy Catherine biztosan elérje a 0-s várost legfeljebb $d + B$ lépésben (ahol d a legrövidebb út hossza, B pedig a megengedett ráhagyás).

Korlátok:

- $2 \leq N \leq 20\,000$, $1 \leq M \leq 20\,000$
- $A \in \{2, 3, 4\}$ (elérhető jelölések száma)
- $B \in \{0, 6, 12, 2N\}$ (megengedett extra lépések száma)